

Universidad de San Carlos de Guatemala.
Centro Universitario de Occidente.
División de Ciencias de la Ingeniería.
Organización de Lenguajes y Compiladores II.
Ing. Mario Tobar.



USAC
TRICENTENARIA
Universidad de San Carlos de Guatemala

MANUAL TÉCNICO

Estudiante	Carné
201931811	Diego José Maldonado Monterroso

Quetzaltenango, Guatemala.

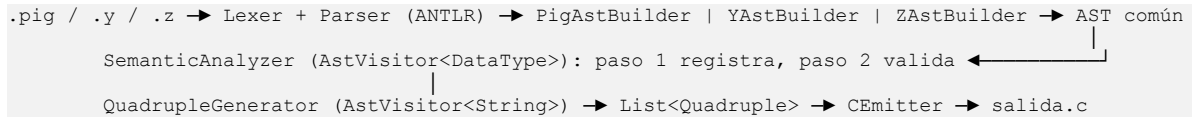
Sábado 26 de septiembre de 2026.

TECNOLOGÍAS

Herramienta	Versión	Uso
Java	21 (compila con JDK 21 o superior)	lenguaje del compilador
ANTLR	4.13.2	lexers y parsers de los 3 lenguajes (visitor, sin listener)
JavaFX	21.0.9	interfaz gráfica
RichTextFX	0.11.3	solo pinta los spans; el color lo decide el lexer propio
Maven	3.9	construcción (<code>antlr4-maven-plugin</code> , <code>javafx-maven-plugin</code>)
gcc	13	compilar el <code>.c</code> generado (en Windows, dentro de WSL)

ARQUITECTURA

Los tres lenguajes producen **el mismo AST**, así el análisis semántico, las cuartetas y el C se escriben una sola vez.



- **MVC:** `view` (JavaFX) no conoce ANTLR; `model` no conoce JavaFX; `controller` los une.
- **CompilerPipeline** resuelve los `import` (relativos a la carpeta del `.pig`), compila cada archivo con su front-end y corre el semántico en **dos pasadas sobre todos los archivos**: primero registra estructuras, clases y firmas; después valida cuerpos. Así `Pila.z` usa `Nodo` sin importarlo, en cualquier orden.
- Corta solo por errores **léxicos**; tras un error sintáctico sigue, para reportar también los semánticos.
- Solo un `.pig` sin errores genera cuartetas y C (un `.y` o `.z` suelto solo se analiza; un `.z` ve las demás clases `.z` de su carpeta, como un paquete de Java). Desde un `.pig`, cada clase que se use se importa, como pidió la auxiliar.

PAQUETES

Paquete	Clases clave	Responsabilidad
model	CompilerPipeline	orquesta el pipeline, imports, devuelve <code>CompilationResult</code>
model.analysis	PigAstBuilder, YAstBuilder, ZAstBuilder, SemanticAnalyzer, ReturnPathChecker	parse tree → AST, validación
model.ast	AstNode, AstVisitor<T>, Program, ClassDeclaration y 38 nodos en total	AST común (Composite + Visitor)
model.symbols	SymbolTable, Scope, VariableSymbol, ArraySymbol, StructSymbol, FunctionSymbol	ámbitos, slots de stack, sobrecarga
model.types	DataType, TypeSystem	tipos y compatibilidad
model.codegen	QuadrupleGenerator, Quadruple, CEmitter	cuartetos y C
model.errors	ErrorManager, CompilerError, SyntaxErrorListener	errores léxicos, sintácticos y semánticos
model.stack	ProcessStackListener, ProcessStep	pila de procesos del parser
view	MainView, EditorTab, HighlightingCodeArea, AstTreeBuilder, ProcessStackPanel	interfaz
controller	ApplicationController, FileController	eventos, archivos y carpetas
util	FileManager, Constants	lectura, escritura y copia en disco; extensiones y título

DIAGRAMA DE CLASES

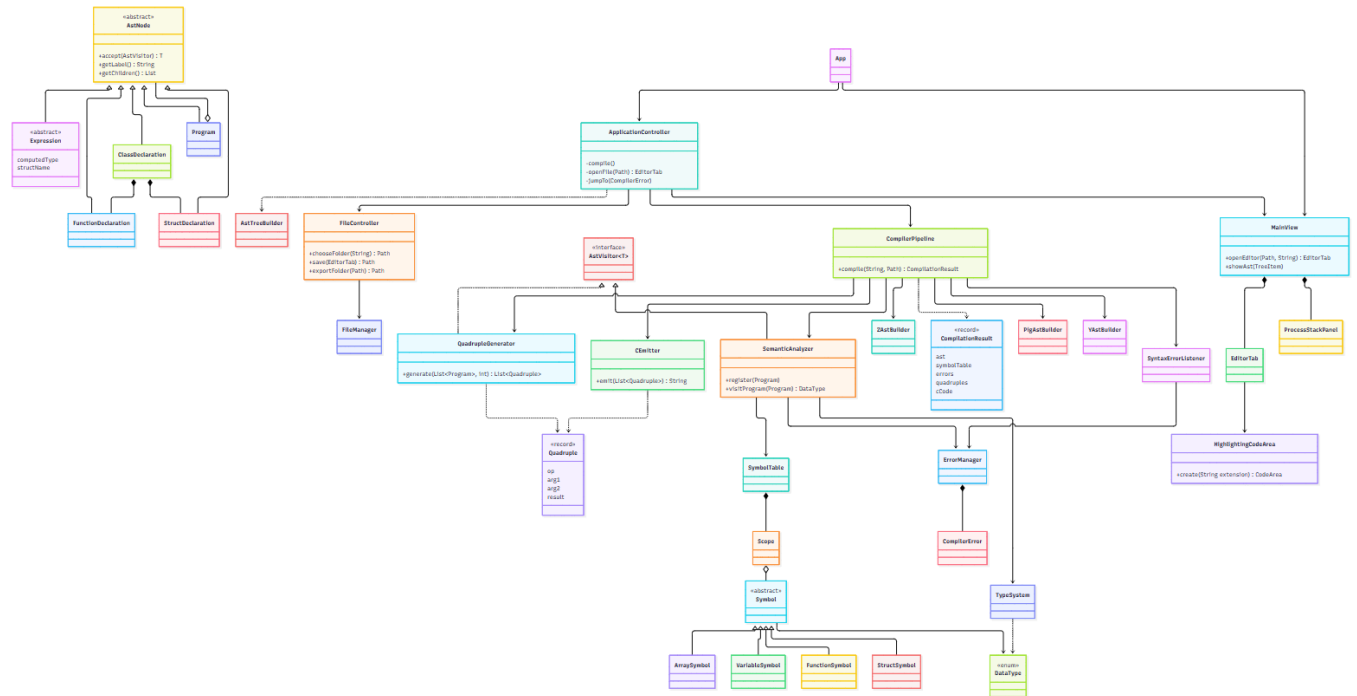


Diagrama de clases

PALABRAS RESERVADAS Y SÍMBOLOS

	PigLatin (.pig)	Y? (.y)	Zetariano (.z)
Estructura	import VARIABLES> VARIABLES[MUNERA> MAIOR> FINIS finis	%estructuras %funciones estructura definir	public private* class void new this*
Declaración	esto series estructura actio ratio novus	— (tipo + nombre)	— (tipo + nombre)
Tipos	numerus decimalis textum littera bool	entero flotante cadena caracter bool	int double String char boolean
Literales	verum falsus	verdadero falso	true false null
Control	si aliter dum facere per perge interrumpe reddere	si entonces sino contrario elegir caso siempre para mientras hacer romper continuar retornar	if else switch case default for while do break continue return
E/S	>> (imprimir) << (leer)	imprimir() leer()	println() print() readln()
Lógicos	&& non	&& !	&& !
Operadores	+ - * / ++ -- == != < > <= >= =	igual + -> (retorno); ++ -- solo como instrucción (i++)	igual + % += -= *= ? :
Comentarios	// /* */ ## ##	// /* */	// /* */

	PigLatin (.pig)	Y? (.y)	Zetariano (.z)
Bloques	{ } + finis;	sangría (INDENT/DEDENT)	{ } (opcionales con una instrucción)

* `private` y `this` se reconocen para reportarlos: el encapsulamiento se contempla en el Proyecto 2.

GRAMATICAS - PRODUCCIONES PRINCIPALES

Completas en `src/main/antlr4/.../grammar/` (PigParser.g4, YParser.g4, ZParser.g4 y sus lexers).

PigLatin

```
programa      : importacion* (VARIABLES declaracionGlobal*)? (MUNERA funcion*)? MAIOR instruccion* FINIS ';'
importacion   : 'import' ID ('.' ID)+ ;?           // el último ID es la extensión (.y / .z)
declaracion   : 'esto' ID ':' tipo expresion? ';' | 'esto' ID ':' 'novus' ID '(' args ')' ';'
               | 'esto' ID ':' tipo '{' valores '}' ';'           // estructura importada de un .y
               | 'series' ID '[' expr ']' ':' tipo '{' valores '}' ';'
funcion       : 'actio' ID '(' params ')' cuerpo 'finis' ';' | 'ratio' tipo ID '(' params ')' cuerpo 'finis'
               ';'
instruccion   : si | dum | facere | per | '>>' expr ('>>' expr)* ';' | destino? '<<' | asignacion | expr ';' |
               ...
si            : 'si' '(' expr ')' bloque ('aliter' '(' expr ')' bloque)* ('aliter' bloque)? 'finis' ';'
dum          : 'dum' '(' expr ')' bloque 'finis' ';'
facere       : 'facere' bloque 'dum' '(' expr ')' ';'
per          : 'per' '(' (declaracion | asignacion) expr ';' actualizacion ')' bloque ('finis' ';')?
```

MUNERA> (funciones propias de PigLatin) es una extensión que se conserva porque los archivos de prueba `.lat` de la auxiliar la usan; el enunciado pide las funciones en archivos `.y`. En una función, las declaraciones van al inicio, dentro de `VARIABLES[ ... ]`.

Y? (la sangría abre y cierra bloques)

```
programa      : ('%estructuras' estructura*)? '%funciones' definicion*
estructura    : 'estructura' ID ':' INDENT (tipo ID ('[' ENTERO ''])?)+ DEDENT
definicion    : 'definir' ID '(' ( '[' | '{' )? tipo ID ',' '*' '(' '-'>' tipo)? ':' bloque
bloque        : INDENT instruccion+ DEDENT
instruccion   : tipo ID ('[' expr '])* ('=' valor)? | si | elegir | para | mientras | hacer | retornar |
               imprimir | ...
si            : 'si' '(' expr ')' 'entonces' bloque ('sino' '(' expr ')' 'entonces' bloque)* ('contrario'
               bloque)?
elegir        : 'elegir' '(' expr ')' '{' ('caso' expr ':' bloque)* ('siempre' ':' bloque)? '}'
para          : 'para' '(' inicio ';' expr ';' actualizacion ')' ':' bloque
mientras      : 'mientras' '(' expr ')' 'hacer' bloque
hacer         : 'hacer' ':' bloque 'mientras' '(' expr ')'
```

La forma `estructura { ... } Nombre`; no es válida en Y? (confirmado por la auxiliar): se reporta con un mensaje propio.

Zetariano

```
programa      : 'public'? 'class' ID '{' miembro* '}'
miembro       : visibilidad? tipo ID ('=' inicializador)? ';'           // atributo
               | visibilidad? ID '(' params ')' bloque                // constructor (sobrecargable)
               | visibilidad? (tipo | 'void') ID '(' params ')' bloque // método (sobrecargable)
tipo          : ('int'|'double'|'char'|'boolean'|'String'|ID) ('[' ']*'
instruccion   : bloque | declaracion ';' | asignacion ';' | 'if' '(' e ')' instr ('else' instr)?
               | 'switch' | 'for' '(' ini? ';' e? ';' act? ')' instr | 'while' | 'do' ... | 'return' e? ';' |
               ...
inicializador : 'new' tipoBase ('[' expr '])+ | '{' elemento (',' elemento)* '}' | expresion
elemento      : '{' elemento (',' elemento)* '}' | expresion           // literales anidados: int[][][] cubo
               = {{1, 2}}
expresion     : or ('?' expresion ':' ternaria)?                       // precedencia de Java
```


TABLA DE COMPATIBILIDAD DE TIPOS

Los tres lenguajes comparten el sistema de tipos; cambia solo el nombre:

Tipo interno	PigLatin	Y?	Zetariano
entero	numerus	entero	int
decimal	decimalis	flotante	double
cadena	textum	cadena	String
carácter	littera	caracter	char
booleano	bool	bool	boolean

Suma + (fila $\oplus$ columna $\rightarrow$ resultado; $\times$ = error semántico)

+	entero	decimal	carácter	booleano	cadena
entero	entero	decimal	entero	$\times$	cadena
decimal	decimal	decimal	decimal	$\times$	cadena
carácter	entero	decimal	entero	$\times$	cadena
booleano	$\times$	$\times$	$\times$	$\times$	cadena
cadena	cadena	cadena	cadena	cadena	cadena

Operación	Regla
- * /	como + pero sin cadenas ni booleanos. <code>entero / entero</code> es división entera (<code>7 / 2 = 3</code>). Un entero que se desborda se imprime, divide y toma residuo como el <code>int</code> de Java (32 bits bajos); es exacto hasta 2^{53}
% (Zetariano)	enteros y caracteres → entero; con decimal → X
< > <= >=	entre numéricos (entero, decimal, carácter) → booleano
== !=	mismo tipo (incluye cadena y booleano) o numéricos entre sí → booleano; objetos/estructuras entre sí o con <code>null</code> → booleano (identidad); las cadenas se comparan por contenido; en Zetariano <code>null</code> solo es igual a <code>null</code> , en PigLatin y Y? una cadena sin valor es ""
&& !	solo booleanos
- unario / ++ --	numéricos / solo entero y decimal
Asignación	mismo tipo, o ensanchando : decimal ← entero o carácter; entero ← carácter; objeto, estructura o cadena ← <code>null</code> . Nunca se estrecha (<code>int x = 2.5</code> es error). Un arreglo completo se comparte por referencia, así que exige el mismo tipo de elemento: un arreglo de enteros no entra donde se espera uno de decimales
Sobrecarga	se elige la versión más específica, como en Java (<code>g('c')</code> prefiere <code>g(int)</code> a <code>g(double)</code>); si dos empatan, la llamada es ambigua y es error
Concatenación	cadena + cualquier primitivo → cadena (<code>"x" + 12 → "x12"</code> ; el booleano sale como <code>verum</code> , <code>verdadero</code> o <code>true</code> según el lenguaje)
Condiciones	<code>si/if</code> , ciclos y ternario exigen booleano

MODELO DE MEMORIA - STACK Y HEAP

```
double stack[1000000], heap[1000000]; double P, H; // P: base del marco actual, H: primera celda libre del heap
```

Qué	Dónde
Globales de VARIABLES>	stack[0 .. G-1]
Marco de una función	stack[P+0] retorno, stack[P+1..n] parámetros, luego locales
Método / constructor	igual, con this en stack[P+1]; los parámetros desde P+2
Cadenas	heap, bytes UTF-8 terminados en -1; los literales se escriben una vez al inicio
Arreglos	heap, aplanados fila por fila : <code>m[i][j] = heap[m + i*columnas + j]</code> ; se pasan por referencia
Estructuras y objetos	heap, un bloque con un campo por celda: <code>obj.campo = heap[obj + índice]</code> ; <code>null = 0</code>

Una llamada evalúa todos los argumentos, los escribe en `stack[P + marco + 1 + i]`, hace `P = P + marco`, llama y restaura `P`: la recursividad funciona por construcción. El runtime detiene el programa con un mensaje (en vez de corromper memoria) ante heap o stack agotado, referencia nula, división entera entre cero y una lectura sin más datos de entrada.

CUARTETAS Y TRADUCCIÓN A C

Cada cuarteta (op, arg1, arg2, resultado) es una línea de C; los temporales t_N son locales de cada función C.

op	Significado	C
+ - * / < > <= >= == !=	aritmética / relacional	<code>t3 = t1 + t2;</code>
div, mod	división y residuo enteros	<code>t3 = intDivide(t1, t2);</code>
=	copia	<code>t1 = t0;</code>
=[] / []=	leer / escribir memoria	<code>t1 = stack[(int) t0]; · heap[(int) t2] = t3;</code>
label, goto, if, ifFalse	control de flujo	<code>L1: · goto L1; · if (!t4) goto L2;</code>
function / end / call / return	funciones	<code>void fn_Pila_apilar__int(void) { ... }</code>
nativas	E/S y cadenas	<code>printString, printInt, readInt, readString, concat, intToString, checkNull, ...</code>

Nombres en C: `fn_<nombre>__<tipos>` por sobrecarga; los métodos llevan su clase:
`fn_Pila_desapilar.`

MANEJO DE ERRORES

Tipo	Detección	Ejemplo de mensaje
Léxico	reglas de error del lexer (TEXTO_SIN_CERRAR, CARACTER_INVALIDO, ...)	<i>Cadena sin cerrar: falta la comilla doble final.</i>
Sintáctico	SyntaxErrorListener (ANTLR en español)	<i>no se esperaba 'opcion', se esperaba {'>', ';'}</i>
Semántico	SemanticAnalyzer	<i>Se retorna 'null' pero el metodo 'obtenerCima' declara 'int'.</i>

Cada error lleva archivo, línea, columna y lexema. Los mensajes nombran los tipos con las palabras del lenguaje del archivo y, cuando aplica, sugieren la corrección (*quite 'private', falta 'FINIS;' al final*).

PRUEBAS

Arnés sin interfaz sobre `CompilerPipeline`, C compilado con `gcc -Wall` (sin warnings) y ejecutado:

Prueba	Resultado
Archivos de Prueba/Pila tal cual	10 errores exactos: <code>private</code> ×2, <code>this</code> ×2, <code>return null</code> , <code>3 ;</code> y <code>FINIS</code> ; faltantes, <code>imprimit</code>
La misma, con las correcciones sugeridas (<code>examples/pila</code>)	0 errores; <code>apila</code> , <code>imprime</code> , <code>desapila</code> ; <code>pila vacía</code> → <i>referencia nula</i>
<code>examples/objetos</code> (ejemplos del enunciado de Zetariano)	salida exacta, incluido <code>dangling else</code> , <code>switch</code> con <code>fallthrough</code> , <code>for(;;)</code> y el cubo <code>int[][][]</code> de Telegram
<code>examples/matrices</code> (literales de Telegram)	salida exacta en PigLatin y Y? (matriz multilínea)
<code>resistencia</code> , enunciado	salida exacta
Los 4 <code>.lat</code> de la auxiliar	<code>helloWorld</code> y <code>ambito_variable_en_funciones</code> : 0 errores, se ejecutan; <code>estructuras</code> : 9 errores; <code>funciones</code> : 7 errores
Telegram: <code>estructura { ... } Nombre;</code> en un <code>.y</code>	1 error sintáctico con mensaje propio
Telegram: <code>Nodo.z</code> en una subcarpeta (o <code>Pila.z</code>), importadas ambas desde el <code>.pig</code>	compila y ejecuta; <code>Pila.z</code> usa <code>Nodo</code> sin importarlo
Bordes	<code>import</code> inexistente, <code>clase</code> ≠ <code>archivo</code> , <code>imports</code> en otro orden, recursión, 20 000 objetos, <code>heap</code> lleno, división entre cero

LIMITACIONES CONOCIDAS

- Zetariano: un arreglo se crea solo en su declaración (`int[] a = new int[n];`); `a = new int[n];` es error sintáctico. No se admiten atributos matriz (`int[][]`), parámetros de dos dimensiones ni métodos que retornen arreglos; en una matriz, las dimensiones internas deben ser constantes (se aplanan con pasos fijos).
- `numerus/entero/int` se guarda como `double` de C: es exacto hasta 2^{53} y el desborde de 32 bits de Java solo se reproduce al imprimir, convertir a texto, dividir o sacar residuo.
- El heap no se libera (1 000 000 de celdas); el control de la pila de C supone 8MB (Linux).